COMP101
Introduction to Programming
2025-26

Lecture-09

Selection

General Overview

Selection: ‘if statement’

- We can execute some lines of code and ignore other lines
  - It alters the ‘flow of control’ (‘flow of execution’)
    - We can take different ‘paths’ each time it executes
    - The program will not fail when it does not execute a line of code

- This is different from sequence statements where . . .
  - Every line of code must execute in the order in which it is coded
  - The program fails if a line of code is not executed

Selection: General forms vs. Python forms

‘General forms’ – common in other languages

- if
- if - then
- if - then - else

‘short-form if’

‘long-form if’

Python general forms don’t use ‘then’

- if
- if - else
- if - elif - else

‘short-form if’

‘long-form if’

‘long-form if’

The if statement must also include a fourth element – a ‘test condition’
The outcome of the test is True or False – this determines which part of the if statement will execute and which part will be ignored

Selection: if - other languages vs Python

C, C#, C++, Java

```
if (condition)   {
   statements          // {} are 'delimeters' to indicate 'scope' or 'body' of the if
}
```

Python

```
if condition:          # 'scope' or 'body' of the construct – uses indentation
   statements             no {} or any other braces to define the scope


if (condition):        #() around the condition are optional - aid readability
   statements          # easier to let Python do a 4-space indent automatically
```

Selection: ‘if’ starts the selection construct

- input statement comes before a selection statement
- ‘if’ follows the input statement as part of a normal sequence
  - it lines up with the input statement
    - ‘if’ as the first word in the code line - Python recognises a selection statement is about to start
- A sub-routine executes to check syntax is correct

```
if (number==6):
```

  - ‘if’ is followed by ‘test condition’ followed by a ‘colon’
  - The parentheses are optional but aid readability

Selection: if statements

```
number = int(input("Enter a number 1 to 6: "))    #we need input for the test

if number == 6:           #test for equality to see whether if should execute
    print ("This indented line executes when input is a 6")
    print ("This line also executes – it is indented under the if")
    print ("We can execute as many lines as needed – must be indented")

print ("Selection ended – this is the first unindented line after the if")
```

Selection: if <test>: - how to construct it

```
#W3L9 dice throws.py
number = int(input(“Enter a throw of a die, 1 to 6: “))     #sequence statement – left aligned
if (number == 6):                                           #sequence statement – left aligned: ‘if’ means start selection
        print ("Throw again")                               #selection statement – 4-space indented under ‘if’
print (“Thank you for playing")                             #sequence statement – left aligned shows end of selection
```

- if (number == 6):
  - starts with reserved word ‘if’ – start selection - obey the syntax of **if statement - test condition - colon**
  - test condition does not have to be in parentheses but makes it easier to read
- print (“Throw again”)
  - indented under ‘if’ hence it belongs to the ‘if statement’ – this is a selection statement
  - indent is 4 spaces done automatically. Best advice – don’t change the indent
  - This statement only executes when user inputs a 6. If user inputs any other number, it is ignored
- The first un-indented statement after the indent shows the selection has ended

Selection: Test condition - Boolean evaluation

The ‘test condition’ evaluates True (T) or False (F)

```
if (test=TRUE):
    execute indented code
next un-indented line executes
```

```
if (test=FALSE):
    ignore indented code
next un-indented line executes
```

```
if (number == 6):
   print ("Throw again")
print (“Thank you for playing")
```

When user inputs a 6 the test=TRUE
and the indented code executes

When any other number is input, the test =FALSE
and the indented code is ignored

Selection: pseudocode if … then … else

1. input (die_number)

2. if (die_number == 6) then
   2.1 print ('throw again')
3. else
   3.1 print ('pass the die')

4. print ('this un-indented line executes after the selection')

These four lines are an 'if-then-else' statement (Python does not use 'then')
If test = TRUE, execute indented code under the if and ignore the else
Start execution at first un-indented line after the selection.
Remember 'else' is part of the selection

If test = FALSE, ignore the if and execute the indented code under the else;
Start execution at first un-indented line after the selection

Only the if will execute (test=True) or the else will execute (test=False)

It is not possible for both of them to execute.
If this does happen, check the indentation

If test = True (user inputs a 6), we get:

throw again
this un-indented line always executes after the selection

If test = False (no input of 6), we get:

pass the die
this un-indented line always executes after the selection

Selection – indentation rules

```
1. input (dice_number)

2. if dice_number = 6 then
       2.1 print (throw again)
3. else
       3.1 print (pass the die)

4. print (this line always executes after the selection)
```

Line-2 and line-3: ‘if’ and ‘else’ align with each other;

Line-4 aligns with line-1, line-2 and line-3
- means the selection statement has finished

Test=True:

i) All indented code under ‘if’ executes sequentially (there can be many lines of indented code)
ii) Look for the next un-indented line; If it starts with ‘else’ – do not execute (because ‘if’ has executed)
iv) Read the next un-indented line of code underneath ‘else’ and execute it

Test=False:

i) Do not execute indented code under ‘if’
ii) Look for the next un-indented line; If it starts with ‘else’, execute all indented code under it (because ‘if’ did not execute)
iii) Read the next un-indented line of code underneath ‘else’ and execute it

Selection – indentation rules

1. input (dice_number)

2. if dice_number = 6 then
    2.1 print ('throw again')
    2.2 print ('that was lucky')
    2.3 print ('do it again!')
3. else
    3.1 print ('pass the die')
    3.2 print ('that was a shame')

4. print ('un-indented line always executes after the selection')

We can execute several indented lines of code under if or else.
All indented code under an if or an else will execute sequentially whenever they 'line up' with each other by indentation - they are in sequence with each other.
Test = TRUE: three print statements under if will execute
Test = FALSE: Two print statements under else will execute

In either case, if–else statements end at the next unindented line that is not if or elif or else

Only the if statement OR the else statement will execute. It can't execute both.
When if executes, Python executes the three indented lines 2.1, 2.2 and 2.3. The first un-indented statement line after these is 'else' – Python will ignore this statement and any indented code under it. It 'goes to' the first unindented line that does not start with 'else'

Selection: No case statement in Python W3L9 months.py

```
month = int(input("Enter a number for a month 1 to 12: "))
if (month == 1):
   print ("31 days")
else:
   if (month == 2):
      print ("2 days")
   else:
      if (month == 3):
         print ("31 days")
      else:
         if (month == 4):
            print ("30 days")
         else:
            if (month == 5):
               print ("31 days")
            else:
               if (month == 6):
                  print ("30 days")
               else:
                  if (month == 7):
                     print ("31 days")
                  else:
                     if (month == 8):
                        print ("31 days")
                     else:
                        if (month == 9):
                           print ("30 days")
                        else:
                           if (month == 10):
                              print ("31 days")
                           else:
                              if (month == 11):
                                 print ("30 days")
                              else:
                                 if (month == 12):
                                    print ("31 days")
```

```
#code extract from left-hand code on slide

month = int(input("Enter a number for a month 1 to 12: "))
if (month == 1):
   print ("31 days")
else:
   if (month == 2):
      print ("28 days")
   else:
      if (month == 3):
         print ("31 days")
      else:
         if (month == 4):
            print ("30 days")
         else:
            if (month == 5):
               print ("31 days")
            else:
               if (month == 6):
                  print ("30 days")
               else:
```

Code analysis shows successful code but incurs a lot of duplication and poor structure

Selection: Logical solution

```
month = int(input("Enter a number for a month 1 to 12: "))
if (month == 1):
   print ("31 days")
else:
   if (month == 2):
      print ("2 days")
   else:
      if (month == 3):
         print ("31 days")
      else:
         if (month == 4):
            print ("30 days")
         else:
            if (month == 5):
               print ("31 days")
            else:
               if (month == 6):
                  print ("30 days")
               else:
                  if (month == 7):
                     print ("31 days")
                  else:
                     if (month == 8):
                        print ("31 days")
                     else:
                        if (month == 9):
                           print ("30 days")
                        else:
                           if (month == 10):
                              print ("31 days")
                           else:
                              if (month == 11):
                                 print ("30 days")
                              else:
                                 if (month == 12):
                                    print ("31 days")
```

```
#tutor lecture-09 dayspermonth.py
#code simplification for duplicated outputs:  4 statements in place of 24 in original
month = int(input('Enter month: '))
if (month==1 or month==3 or month==5 or month==7 or month==8 or month==10 or month==12):
   print ('Days = 31')

elif (month==4 or month==6 or month==9 or month==11):
   print ('Days = 30')

else:
   print ('Days = 28 February non-leap-year')

'''
code analysis shows only 3 possible outcomes
Use compound expressions to handle the logic
Initial draft - no val
idation hence out-of-range i/p trips the else exception
'''
```

Triple-apostrophe commentary outputs in green text in IDLE.
In red here for clarity

Don’t do this

```
#input odd months 1,3,5 and even months 2,4,6 to determine month length
input (month)
if month = 1 then:
        days = 31
        else:
                if month = 3 then:
                        days = 31
                        else:
                                if month = 5 then:
                                        days = 31
else:
        if month = 2 then:
                days = 28
        else:
                if month = 4 then:
                        days = 30
                else:
                        if month = 6 then:
                                days = 30
ifend
```

This pseudocode design is for the first 6 months of the year (for space issues on the slide).

It is a poor approach – both logic-wise and implementation-wise.

There is a lot of duplication (even more if we were to include the other 6 months), it is hard to read, it is hard to understand, it is unbalanced

Selection: Case/Switch (not in Python)

- A ‘case’ or ‘switch’ statement provides a short-hand way to manage many if-then-else statements
  - Can help issues of long, difficult to read and maintain selection code

```
input (month)
CASE month OF
    1,3,5,7,8,10,12: days = 31
    4,6,9,11       : days = 30
    2              : days = 28
End
```

Code analysis shows successful code with no duplication and a better structure.

PYTHON DOES NOT SUPPORT THE CASE/SWITCH STATEMENT

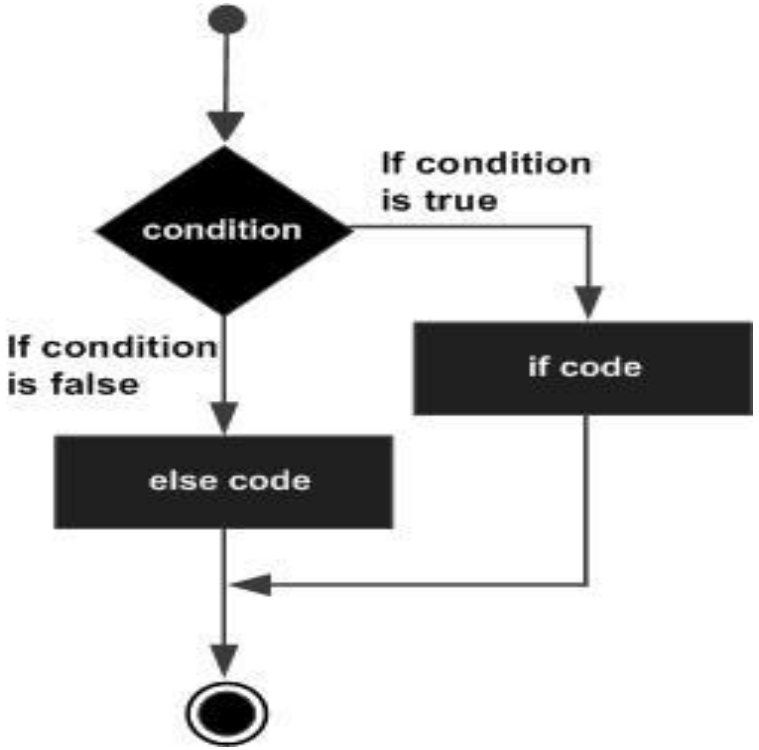
It uses an if-elif-else structure instead

Selection: Terminology

- ‘Branching’, ‘Conditional branching’
- Keywords: if – then – else
  - no ‘then’ in Python – use ‘elif’
- Conditional test (Boolean test)
  - True or False (T/F)
- Comparison test (binary) operators
  - < <= > >= <>or (!=) ==
- Logical Operator test (Boolean test)
  - and, or ,not (all give True or false)
  - xor – ‘exclusive or’

Selection: Flow chart

- Testing is more complex – test all paths in the selection



test for condition = True
and then
test for condition = False (i.e. the else part)

This can become complex when we have nested if statements